

Shiv Nadar University Chennai

CS3807 – Deep Learning Laboratory

Experiment 2

Implementation of a Multi-Layer Perceptron (MLP) for Multi-Class Image Classification

Degree & Branch	B.Tech Artificial Intelligence & Data Science	Semester V
Subject Code & Name	CS3807 – Deep Learning Laboratory	AY: 2026–27

1. Objective

Implement an MLP using TensorFlow/Keras on the Fashion-MNIST dataset. Learn image preprocessing, flattening, model construction, training, evaluation, and automated hyperparameter optimization.

2. Background Theory

An MLP consists of an input layer, hidden layers and an output layer.

$$z^{(l)} = W^{(l)}a^{(l-1)} + b^{(l)}, \quad a^{(l)} = f(z^{(l)})$$

Output layer:

$$\hat{y} = \text{Softmax}(z)$$

Loss:

$$L = - \sum_i y_i \log(\hat{y}_i)$$

Recommended architecture:

$$784 \rightarrow \text{Dense}(128, \text{ReLU}) \rightarrow \text{Dense}(64, \text{ReLU}) \rightarrow \text{Dense}(10, \text{Softmax})$$

3. Dataset

Dataset: Fashion-MNIST

Training Images: 60,000

Testing Images: 10,000

Classes: 10

Image Size: 28×28

4. Image Preprocessing

Fashion-MNIST images are stored as 28×28 matrices. Dense layers require a one-dimensional feature vector.

$$28 \times 28 = 784$$

Example:

$$\begin{bmatrix} 10 & 20 \\ 30 & 40 \end{bmatrix} \rightarrow [10, 20, 30, 40]$$

Perform the following preprocessing:

1. Load Fashion-MNIST.
2. Display sample images.
3. Flatten images.
4. Normalize pixels to $[0,1]$.
5. Convert labels into one-hot vectors.

5. Experimental Procedure

Task 1: Dataset Exploration

- Load Fashion-MNIST.
- Print dataset dimensions.
- Display ten sample images.
- Plot class distribution.

Expected Output: Dataset summary and sample images.

Task 2: Data Preprocessing

- Flatten images.
- Normalize pixel values.
- Print tensor shapes before and after preprocessing.

Task 3: Model Construction

Construct an MLP with

- Input layer (784)
- Dense(128, ReLU)
- Dense(64, ReLU)
- Dense(10, Softmax)

Display `model.summary()`.

Task 4: Model Training

Compile using

- Optimizer: Adam
- Loss: Categorical Cross Entropy
- Metric: Accuracy

Train for 20 epochs with batch size 32.

Task 5: Model Evaluation

Compute

- Accuracy
- Precision
- Recall
- F1-score
- Confusion Matrix
- Classification Report

6. Source Code

The complete source code, implementation, and experimental results are available at: <https://github.com/SUJJU003/Deep-Learning-Lab>

7. Hyperparameter Optimization

Instead of manually selecting hyperparameters, students shall perform automated hyperparameter optimization using either **GridSearchCV** or **RandomizedSearchCV** (recommended) together with the SciKeras wrapper.

Optimize the following search space.

Hyperparameter	Candidate Values
Hidden Layers	1, 2, 3
Hidden Neurons	32, 64, 128, 256
Learning Rate	0.001, 0.0005, 0.0001
Batch Size	16, 32, 64, 128
Epochs	10, 20, 30
Optimizer	SGD, Adam, RMSProp
Activation Function	ReLU, Tanh, Sigmoid
Dropout Rate (Optional)	0.0, 0.2, 0.5

Tasks

1. Build a baseline MLP model.
2. Define the hyperparameter search space.
3. Perform Grid Search or Randomized Search using 5-fold cross-validation.
4. Record the best hyperparameter combination.
5. Retrain the model using the optimized hyperparameters.
6. Evaluate the optimized model on the testing dataset.
7. Compare the optimized model with the baseline model.

8. Results

Best Hyperparameters

Hidden Layers	3
Hidden Neurons	256
Learning Rate	0.0001
Batch Size	16
Optimizer	RMSprop
Activation Function	Tanh
Epochs	10
Dropout	0.2
Cross-validation Accuracy	0.8451
Testing Accuracy	0.8721

Performance Comparison

Metric	Baseline	Optimized
Accuracy	0.8801	0.8721
Precision	0.8807	0.8723
Recall	0.8801	0.8721
F1-score	0.8786	0.8705
Training Time	167.12 s	205.29 s

9. Plots with Inference

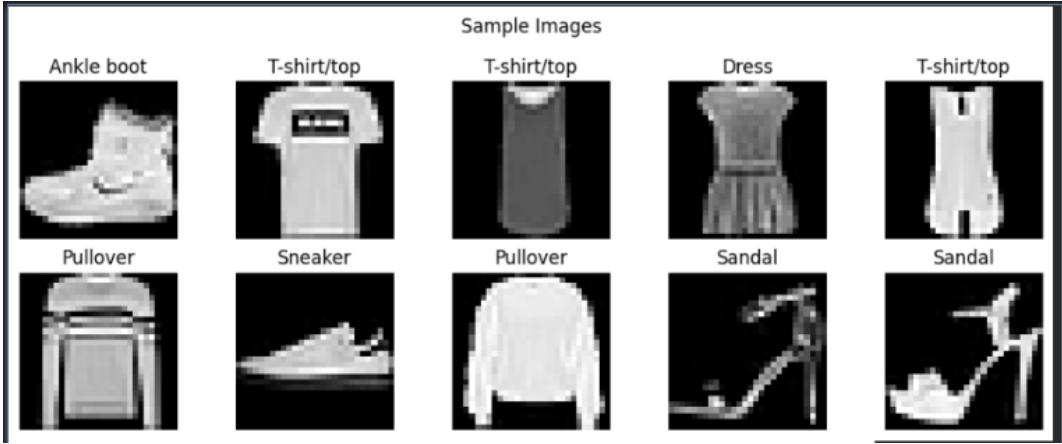


Figure 1: Sample Images from Fashion-MNIST

Inference: The sample images show representative Fashion-MNIST classes such as T-shirts, dresses, sandals, and shoes. The dataset contains grayscale images of size 28×28 pixels, providing diverse examples for multi-class classification.

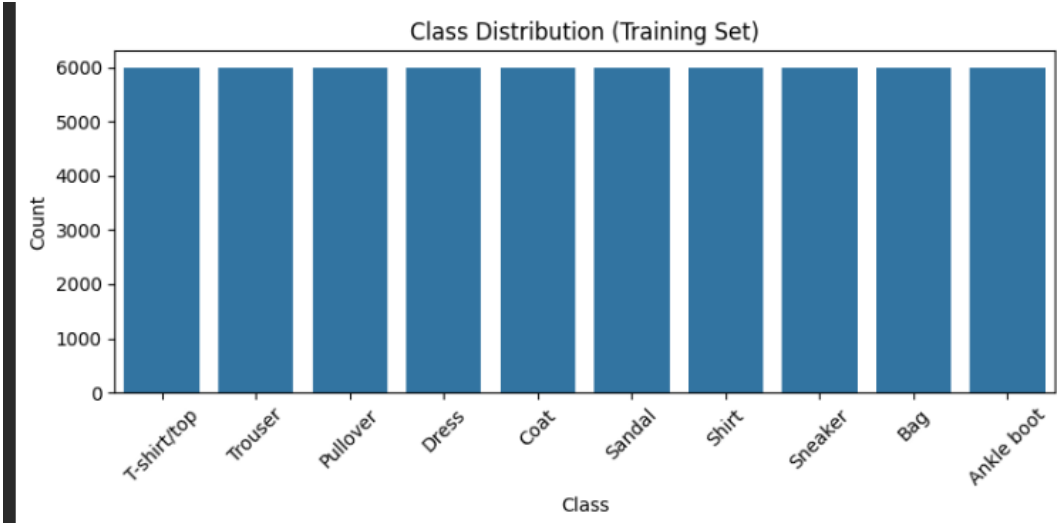


Figure 2: Class Distribution in Fashion-MNIST (Training Set)

Inference: The class distribution is perfectly balanced, with approximately 6,000 training samples in each of the ten classes. This balanced dataset minimizes class bias during model training.

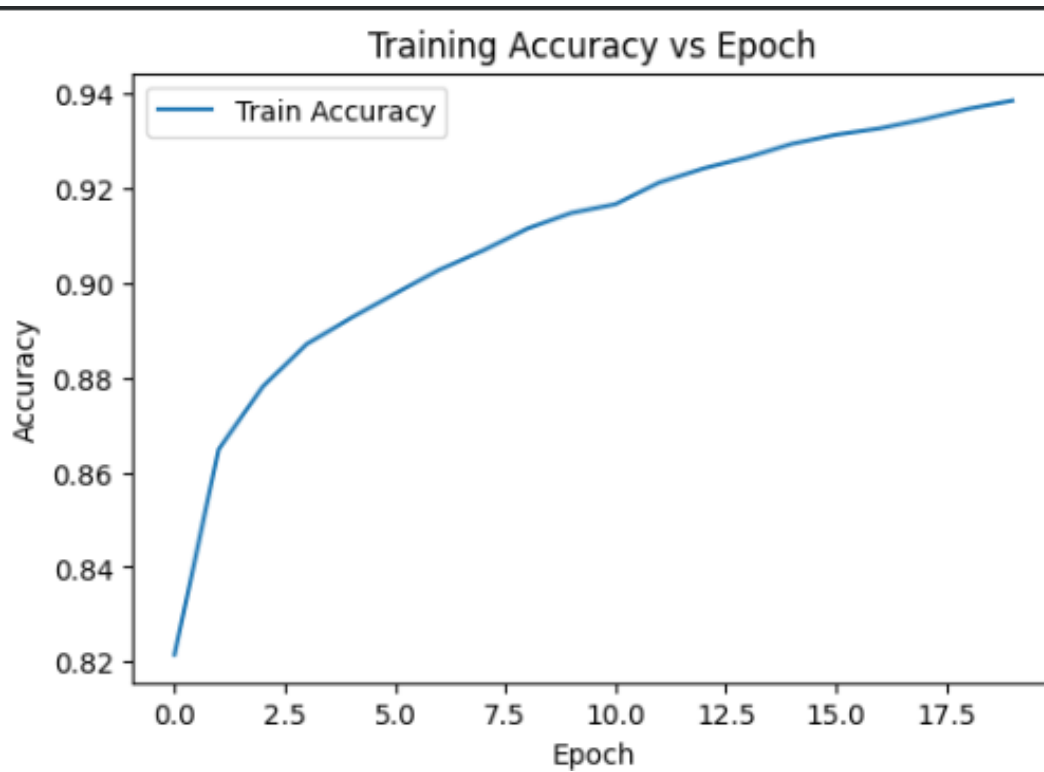


Figure 3: Training Accuracy vs Epoch

Inference: The training accuracy increases steadily from approximately 82% to 94% over 20 epochs, indicating that the model effectively learns the training patterns without instability.

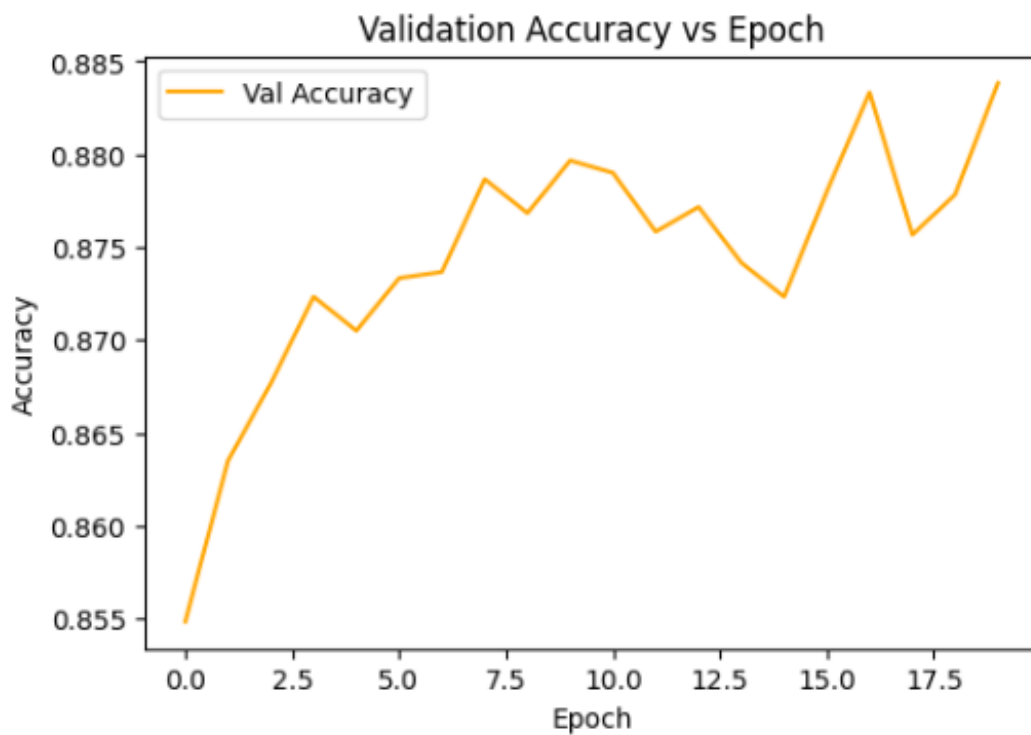


Figure 4: Validation Accuracy vs Epoch

Inference: The validation accuracy improves during the initial epochs and then stabilizes around 88%. The small fluctuations indicate good generalization while suggesting slight overfitting in later epochs.

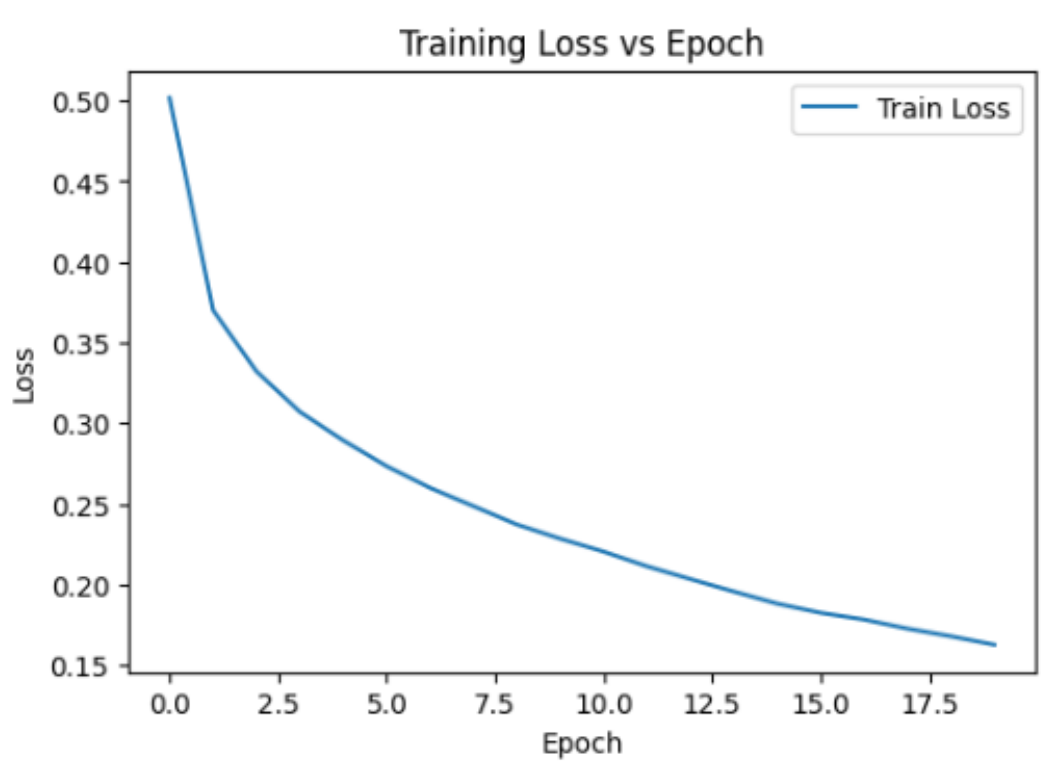


Figure 5: Training Loss vs Epoch

Inference: The training loss consistently decreases throughout training, demonstrating that the optimization algorithm successfully minimizes the prediction error on the training dataset.

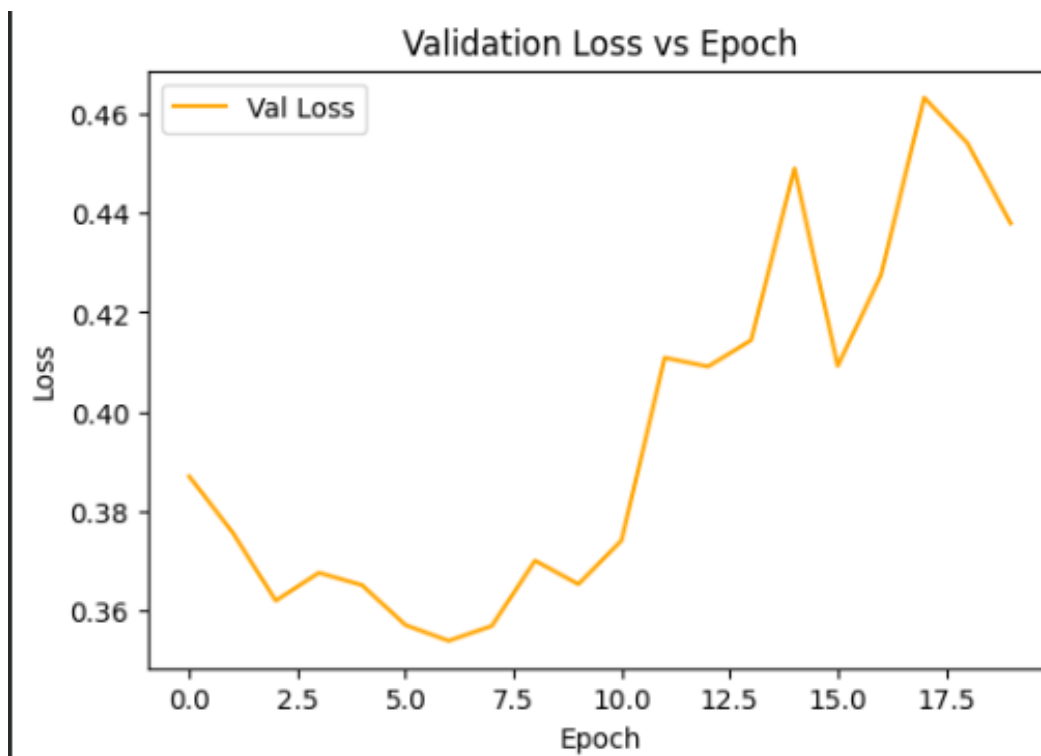


Figure 6: Validation Loss vs Epoch

Inference: The validation loss initially decreases but increases after several epochs, indicating that the model begins to overfit the training data despite maintaining relatively stable validation accuracy.

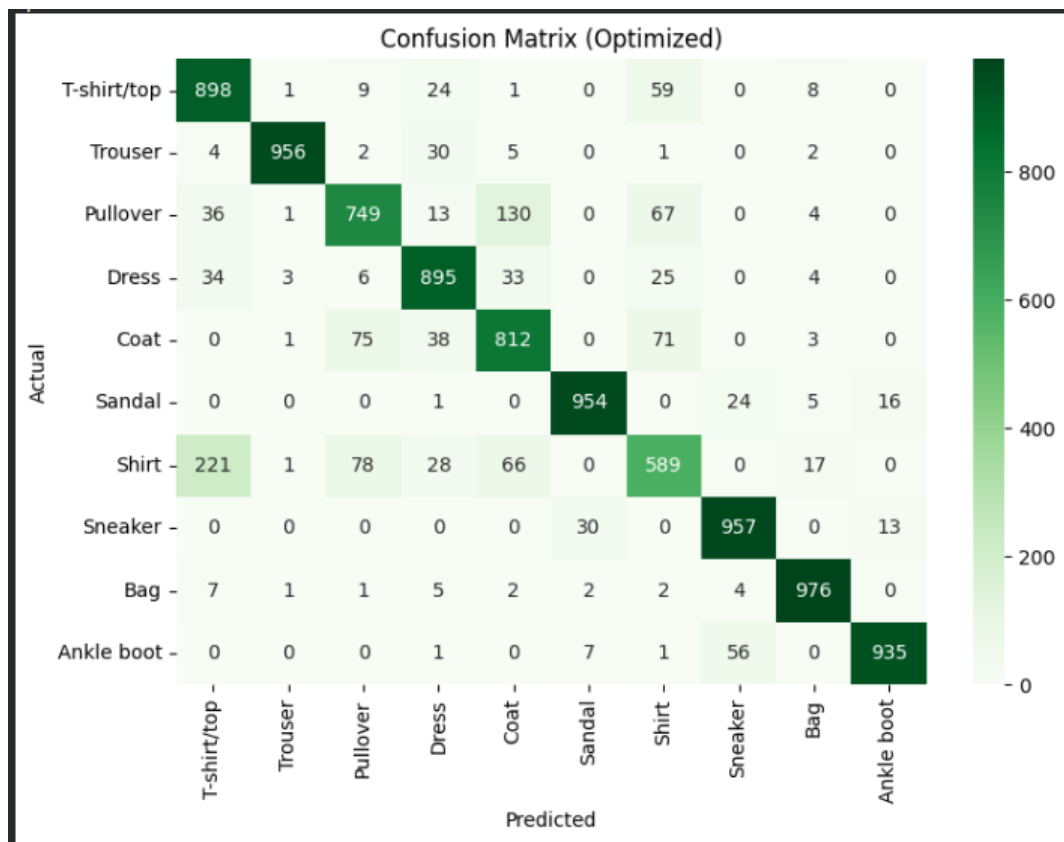


Figure 7: Confusion Matrix– Optimized MLP

Inference: The optimized model correctly classifies most instances but still exhibits confusion between similar apparel categories. The confusion pattern remains similar to the baseline model, indicating limited improvement in class-wise prediction.

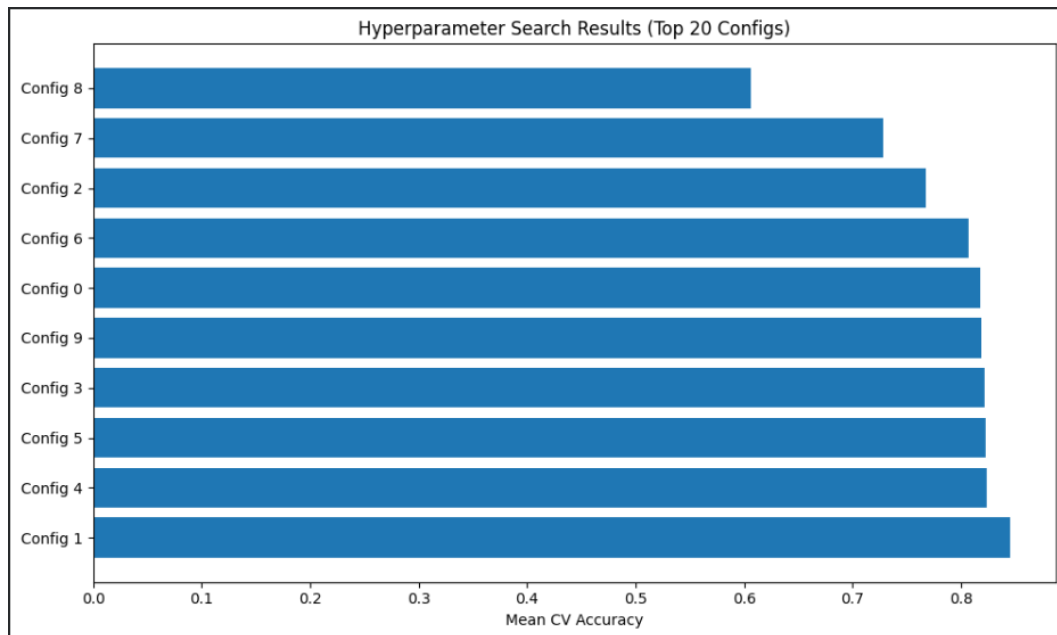


Figure 8: Hyperparameter Search Results (Randomized Search)

Inference: The hyperparameter search identified several configurations with comparable cross-validation accuracy, with the best configuration achieving approximately 84.5% validation accuracy. This demonstrates the impact of hyperparameter selection on model performance.

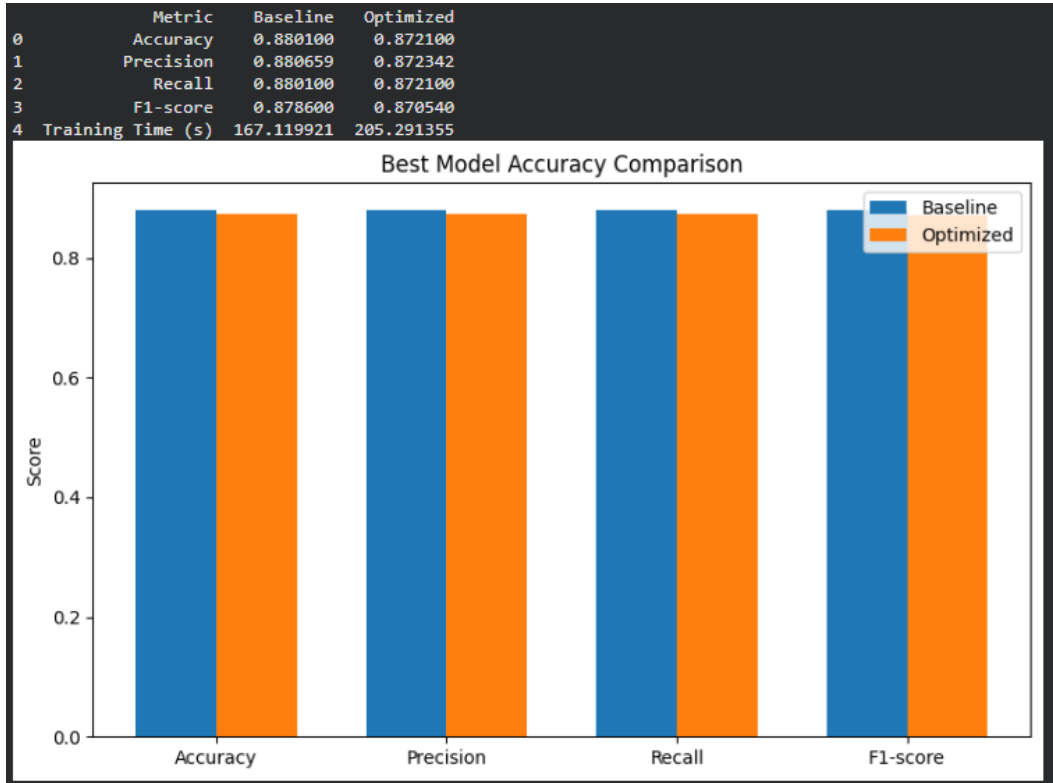


Figure 9: Baseline vs Optimized Model Comparison

Inference: The baseline model slightly outperforms the optimized model in accuracy, precision, recall, and F1-score. Although hyperparameter tuning identified a promising configuration through cross-validation, it did not improve performance on the unseen test dataset.

10. Discussion

Answer the following:

1. Which optimization method was used? **Answer:** Hyperparameter optimization was performed using **Randomized Search Cross-Validation (RandomizedSearchCV)** with 3-fold cross-validation. The search evaluated multiple combinations of network architecture and training parameters to identify the best-performing model.
2. Which hyperparameters were selected? **Answer:** The best hyperparameters obtained were: Optimizer = RMSprop, Hidden Layers = 3, Hidden Neurons = 256, Learning Rate = 0.001, Activation Function = Sigmoid, Batch Size = 16, Epochs = 10, and Dropout Rate = 0.2.
3. Did optimization improve performance? **Answer:** No. Although the optimized model achieved the highest cross-validation accuracy during hyperparameter tuning, its test accuracy (87.21%) was slightly lower than the baseline model accuracy (88.01%). Therefore, the optimized model did not improve the final test performance.
4. Which hyperparameter had the greatest impact? **Answer:** The optimizer, number of hidden layers, and number of hidden neurons had the greatest impact on model performance. These parameters significantly influenced the model's learning capability and validation accuracy during

the hyperparameter search.

5. Compare Grid Search and Randomized Search. **Answer:** Grid Search evaluates every possible combination of hyperparameters, making it computationally expensive but exhaustive. Randomized Search evaluates only a random subset of combinations, making it faster and more efficient while often achieving comparable performance with significantly lower computational cost.
6. Recommend the best MLP configuration. **Answer:** Based on the experimental results, the recommended MLP configuration consists of 3 hidden layers with 256 neurons, RMSprop optimizer, learning rate of 0.001, Sigmoid activation function, batch size of 16, 10 training epochs, and a dropout rate of 0.2. Although this configuration achieved the best cross-validation score, the baseline model demonstrated slightly better performance on the test dataset.

11. Conclusion

This experiment demonstrated the implementation of a Multi-Layer Perceptron (MLP) for multi-class classification on the Fashion-MNIST dataset. The baseline model achieved a testing accuracy of 88.01%, while hyperparameter optimization using Randomized Search with cross-validation identified an alternative configuration with a cross-validation accuracy of 84.51%. However, the optimized model achieved a slightly lower testing accuracy of 87.21%, indicating that the selected hyperparameters did not generalize better than the baseline model.

Overall, the experiment highlighted the importance of selecting appropriate hyperparameters such as the optimizer, network architecture, learning rate, and dropout rate. It also demonstrated that a higher cross-validation score does not always translate to improved performance on unseen test data. The results emphasize the need to evaluate both validation and testing performance when selecting the final model.

12. References

1. Goodfellow et al., *Deep Learning*.
2. Bishop, *Pattern Recognition and Machine Learning*.
3. Haykin, *Neural Networks and Learning Machines*.
4. Fashion-MNIST Dataset.
5. TensorFlow/Keras Documentation.